

JVD Internal Management System

Team Workflow — Developers, Branches & Task Ownership · v1.0 FINAL · 2026-07-04

JVD Team Workflow — Developers, Branches & Task Ownership

Date: 2026-07-05 · Part of the v1.0 doc set · Companion to `IMPLEMENTATION_ROADMAP.md`

1. The team

Dev	Role	Primary lane	Also handles
Val	Team lead · main backend	Money paths, data integrity, architecture, releases & deploys	Final review on anything financial
Emman	Backend · QA	Platform (queues, CI, notifications, API layer), test suite ownership	QA sign-off on every phase
Greg	Frontend	Design system: tokens, component library, dashboards	Backend endpoints for his own features when needed
Jerald	Frontend	Data layer (React Query), checkout wizard, page migrations	Repo hygiene, help center

Everyone can cross lanes (you're all full-stack in practice) — the lanes exist so **two people rarely edit the same files**, which is what makes merging easy. Capacity note: Val plans at ~60% coding capacity; lead/review/deploy duties consume the rest — don't schedule Val as a full IC.

Review rule: every PR is reviewed by someone who didn't write it — backend PRs by Val or Emman, frontend PRs by Greg or Jerald, cross-stack by anyone. Anything touching invoices, payments, ledger, or payroll **must** have Val's review.

2. Task ownership matrix (Owner drives & merges; Support pairs/reviews)

Phase 0 — Stop the bleeding

Item	Task	Owner	Support
0.1	Payroll tax test failure (investigate + fix)	Emman	Val
0.2	<code>VisaProcessing.tsx</code> TS error	Greg	—
0.3	<code>DB::transaction</code> on money paths	Val	Emman
0.4	Repo cleanup + <code>.gitignore</code>	Jerald	Val
0.5	Branch migration + protected <code>main</code> (§4 below)	Val	—
0.6	Docs consolidation	Jerald	—

Phase 1 — CI/CD + environments

Item	Task	Owner	Support
1.1	GitHub Actions (backend + frontend)	Emman	Val
1.2	Docker compose + prod Dockerfile	Val	Emman
1.3	Staging + prod servers, deploy pipeline	Val	Emman
1.4	Sentry + uptime monitoring	Emman	—
1.5	Backups + restore drill	Val	Emman
1.6	Prod env hardening	Emman	—
1.7	Sales domain workshops	Val	Jerald (he builds the checkout later — he should hear the domain firsthand)

Phase 2 — Backend correctness & architecture

Item	Task	Owner	Support
2.1	Queues live	Emman	—
2.2	Derived payment status, locks, idempotency	Val	Emman
2.2b	DB integrity wave (cascades→restrict, FKs, CHECKs)	Val	—
2.2c	Merge travel tables + partial unique indexes	Val	Emman
2.3	Abilities & roles-as-data	Emman	Val
2.4	Workflow engine + approvals inbox	Val	Emman
2.5	Ledger postings + finalization snapshots	Val	Emman
2.5b	Split bookings out of invoices	Val	Emman
2.6	Notifications + email digests	Emman	Greg (bell UI)
2.7	Document repository (DMS)	Emman (BE)	Greg (FE module)
2.8	FormRequests, Resources, pagination, /api/v1	Emman	Val
2.9	Thin god controllers (Val: Billing, TripTicket, PublicRequestAction · Emman: ProcurementDocument, Dashboard, PassportCase)	Val + Emman	—
2.10	Money-path test expansion	Emman	Val

Phase 3 — Frontend rebuild

Item	Task	Owner	Support
3.1	Design tokens + ESLint color rule	Greg	—
3.2	Component library (Greg: AppShell, DataTable, Drawer, StatusPill, StatCard · Jerald: Modal/wizard, ListRow, CommandPalette, Toast, EmptyState, SharePopover)	Greg + Jerald	—
3.3	Surface rules migration (overlays → shared components)	Greg	Jerald
3.4	React Query everywhere + code splitting	Jerald	—
3.5	Dashboards-as-widgets	Greg (FE)	Emman (widget endpoints)
3.6	Service catalog + checkout wizard	Jerald (FE)	Val (catalog BE)
3.7	Page migrations — Greg: Dashboards, HR, Admin, Accounting · Jerald: Sales, Travel, Logistics, Inventory, Operations	Greg + Jerald	—
3.8	Auth/login UX polish	Greg	—

Phase 4 / 4.5 — Proof & rollout

Item	Task	Owner	Support
4.1	Vitest + RTL on component library	Greg + Jerald	—
4.2	Playwright E2E suite	Emman	all
4.3	k6 load test	Val	Emman
4.4	CI coverage gates	Emman	—
4.5.1	Dependency CVE patches + audit in CI	Emman	—
4.5.2	Runbook + access/env inventories	Val	—
4.5.3	Help center + FAQ + "Report a problem"	Jerald	Emman (IT-request workflow)
4.5.4	In-app onboarding (checklists, tooltips)	Greg	—
4.5.5	Pilot, champions, training sessions	Val	all
4.5.6	DPA/BIR compliance items	Val	—

Parallelism guide: Phase 0 items are all independent — all four of you start day one. From Phase 2/3, the standing formation is: Val + Emman on backend items in roadmap order, Greg + Jerald on frontend items in roadmap order, syncing at the integration points marked as Support above.

3. Branch strategy — trunk-based, item-numbered

One rule: `main` is the only long-lived branch. Everything else lives days, not weeks.

- **Naming:** `<type>/<roadmap-item>-<slug>` → `fix/0.1-payroll-tax`, `feat/2.4-workflow-engine`, `chore/0.4-repo-cleanup`. The item number makes branch ↔ task ↔ PR traceable at a glance. Big items ship as several branches: `feat/2.4-workflow-engine-schema`, `feat/2.4-workflow-engine-service`, ...

- **Lifecycle:** branch from fresh `main` → small commits → PR early (draft is fine) → CI green + review → **squash-merge** → delete branch. Squash keeps `main` history one-commit-per-PR, readable and revertible.
- **Stay current:** `git pull origin main` into your branch **daily** (merge, not rebase — simpler for the team, no force-pushes). Small PRs ($\leq \sim 400$ lines) + daily syncs + lane ownership = merge conflicts mostly disappear.
- **PR description:** roadmap item number, what changed, how you verified it. That's it.
- **No more personal branches.** `Emman`, `Val`, `Greg`, `greg_Backend` are retired by the migration below. Branches are named after *work*, not *people* — that's what was making merging painful: personal branches accumulate weeks of divergence; task branches can't.

4. One-time migration (Val runs this — ~15 minutes)

The analysis is already done and it's the best case: `Emman` is 190 commits ahead of `main`, and `main` / `origin/Val` / `origin/Greg` contain zero unique commits. So `Emman` is simply promoted to be the new `main` — a fast-forward, no merge conflicts possible.

```
# 0. On branch Emman: deal with the uncommitted WIP first
#   (DashboardController.php edit + untracked test*.php files)
git add backend/app/Http/Controllers/DashboardController.php
git commit -m "wip: dashboard changes"          # or discard if abandoned — Emman decides
# test*.php files: do NOT commit — item 0.4 deletes them

# 1. Promote Emman to main (pure fast-forward)
git checkout main
git merge --ff-only Emman
git push origin main

# 2. Retire the personal branches
git push origin --delete Emman Val Greg greg_Backend
git branch -d Emman Val
```

Then protect `main` on GitHub — this changes repository access settings, so it's yours to do, not mine: GitHub → repo **Settings** → **Branches** → **Add branch ruleset** (or classic protection rule) on `main`: ☒ Require a pull request before merging (1 approval) · ☒ Require status checks to pass (select the CI jobs once 1.1 lands) · ☒ Block force pushes. Note: on a **private** repo, branch protection needs GitHub Pro/Team — if the repo is private on a free plan, either make it public, upgrade, or enforce the rule socially until then (CI still blocks bad merges once required checks exist).

5. Worktrees — work on two things without stash hell

Git worktrees give each branch its own folder — no stashing, no half-done checkouts. Since you all cross between frontend and backend, this is the comfortable setup. Each dev, on their own machine:

```
# One-time: keep your main clone as the "main" worktree, then e.g.:
git worktree add ../jvd-feature feat/2.4-workflow-engine # your current task
git worktree add ../jvd-review -b review # for checking out PRs to review

# Review a PR without touching your work:
cd ../jvd-review && git fetch origin && git checkout feat/2.3-abilities

# When a branch is merged and deleted:
git worktree remove ../jvd-feature
git worktree list # see what you have
```

Practical layout per dev: Desktop/JVD-Internal-Management-System (main, always clean) · Desktop/jvd-feature (your task) · Desktop/jvd-review (PR reviews). Each worktree needs its own backend/.env and vendor/ + node_modules installs (or point them at the same DB — fine locally). Never two worktrees on the same branch.

6. The weekly rhythm

- **Monday 15 min:** pick roadmap items per the matrix, confirm nobody's lanes collide this week.
- **PRs reviewed within 24h** — a waiting PR blocks a teammate; review beats new code.
- **Friday:** anything merged? Val tags/deploys staging (auto once 1.3 lands). Demo to each other — 10 minutes, keeps everyone seeing the whole system.
- **Roadmap checkboxes are the status report** — tick them in the PR that completes them; no separate tracker needed at this team size.